Question 5 DAA Project

Sayan Bhadra(*bmat2429*)
Subhojit Maji(*bmat2434*)

27th October 2026

1 Introduction and Problem Statement

In organizational network design, a central challenge is selecting a minimal supervisory or coordination team such that every member of the organization remains within a fixed proximity to at least one coordinator. An organization is modeled as a rooted tree $T = (V, E)$, where V represents the set of employees, $|V| = n$, and E represents the set of managerial direct-report links, $|E| = n - 1$. The chief executive officer (CEO) occupies the root vertex $r \in V$.

The path distance between any pair of nodes $u, v \in V$, denoted $\text{dist}_T(u, v)$, is defined as the unique number of edges connecting u and v in T. We seek to determine a subset of vertices $S \subseteq V$ of minimum cardinality such that every employee in V is within path distance at most 2 from some member of S. Mathematically, the problem is stated as:

$$\text{minimize } |S| \quad \text{subject to} \quad \forall u \in V,\ \exists v \in S \text{ such that } \text{dist}_T(u, v) \leq 2.$$

2 Observations

To construct a basis for the algorithm, we begin by observing where the most constrained nodes lie in a rooted tree.

Consider a leaf node $u \in V$. Because u has no descendants, its ability to reach other nodes is strictly limited to upward traversal toward its parent $p(u)$, its grandparent $g(u)$, its siblings, and their immediate children. If u is not covered by any previously selected team member, u *must* be covered by adding some vertex w to S such that $\text{dist}_T(u, w) \leq 2$.

Evaluating Candidate Placements
Since u is a leaf, any valid vertex w that satisfies $\text{dist}_T(u, w) \leq 2$ must belong to one of four topological categories:

1. **The leaf u itself:** Placing a team member at u covers u, its parent $p(u)$, and its grandparent $g(u)$. However, it provides zero coverage to any nodes adjacent to $g(u)$ or above $g(u)$.
2. **The parent $p(u)$:** Placing a team member at $p(u)$ covers u, all siblings of u, $p(u)$ itself, $g(u)$, and the siblings of $p(u)$.
3. **The grandparent $g(u)$:** Placing a team member at $g(u)$ covers u, $p(u)$, $g(u)$, all children and grandchildren of $g(u)$, as well as the great-grandparent $g(g(u))$ and great-great-grandparent $g(g(g(u)))$.
4. **A sibling of u:** Placing a team member at a sibling of u covers u via path length 2 through $p(u)$, but offers no upward reach beyond $g(u)$.

The Greedy Insight

Comparing these candidates reveals a strict containment hierarchy of covered sets. Any uncovered node higher up in the tree or in neighboring subtrees that could possibly be reached by choosing u or $p(u)$ is also reached by choosing $g(u)$. In addition, $g(u)$ reaches nodes two levels higher than $p(u)$ can reach.

Consequently, whenever a leaf (or deepest uncovered node) requires coverage, selecting its **grandparent** (or highest available ancestor if the node is at depth < 2) strictly dominates all other choices. Processing nodes in reverse order of depth (bottom-up) guarantees that we never waste coverage capacity on lower nodes when an ancestor can cover them alongside unvisited upper nodes.

3 Pseudo Code

Following the observations above, the algorithm operates in three main steps:

1. **Depth Evaluation:** Compute the depth of each node relative to the root r using a Breadth-First Search (BFS) or Depth-First Search (DFS), placing r at depth 0.
2. **Topological Ordering:** Sort all vertices in descending order of their depth.
3. **Bottom-Up Greedy Selection:** Iterate through the sorted vertices. For each node u, if u is marked as uncovered:
   - Determine target node v: if $g(u)$ exists, set $v = g(u)$; else if $p(u)$ exists, set $v = p(u)$; else set $v = u$.
   - Insert v into S.
   - Traverse the distance-2 neighborhood of v and mark all encountered vertices as covered.

Algorithm 1 Minimum Distance-2 Dominating Set on a Tree

Require: Rooted Tree $T = (V, E)$ with root r
Ensure: Minimum dominating set $S \subseteq V$

```
Compute depth[u] and parent pointers p[u] for all u ∈ V via BFS from r
L ← list of vertices sorted by depth in descending order
S ← ∅
covered[u] ← False for all u ∈ V
for each u ∈ L do
    if not covered[u] then
        v ← u
        if p[v] ≠ NULL then
            v ← p[v]
            if p[v] ≠ NULL then
                v ← p[v]                          ▷ Select grandparent if available
            end if
        end if
        S ← S ∪ {v}
        for each w ∈ V with dist_T(v, w) ≤ 2 do
            covered[w] ← True
        end for
    end if
end for
return S
```

4 Analysis of Algorithm

Let $T = (V, E)$ be a tree with $|V| = n$ and $|E| = n - 1$. For any node $v \in V$, let $N(v) = \{w \in V \mid (v, w) \in E\}$ denote its open 1-neighborhood, and $\deg(v) = |N(v)|$. The closed distance-2 neighborhood of v is:

$$N_2[v] = \{w \in V \mid \text{dist}_T(v, w) \leq 2\} = \{v\} \cup N(v) \cup \left(\bigcup_{x \in N(v)} N(x) \right).$$

Time Complexity Analysis
The total execution time $T(n)$ of the algorithm is decomposed into three distinct phases:

$$T(n) = T_{\text{depth}}(n) + T_{\text{sort}}(n) + T_{\text{greedy}}(n).$$

Computing depths and parent pointers via BFS requires visiting each node once and inspecting each incident edge twice. Thus:

$$T_{\text{depth}}(n) = O(|V| + |E|) = O(n + n - 1) = O(n).$$

Since node depths are integers bounded in the range $[0, n - 1]$, we employ Bucket Sort (or group vertices into depth lists during BFS and read in reverse). The sorting step takes linear time:

$$T_{\text{sort}}(n) = O(n).$$

The outer loop executes n times. Evaluating `if not covered[u]` takes $O(1)$ time per node. When a node v is added to S, we perform a local BFS up to distance 2 to mark $N_2[v]$. The number of operations required to explore $N_2[v]$ is proportional to the sum of degrees of v and its immediate neighbors:

$$\text{Cost per inclusion of } v = O\left(1 + \deg(v) + \sum_{x \in N(v)} \deg(x)\right).$$

Summing this cost over all chosen nodes $v \in S$:

$$T_{\text{greedy}}(n) = \sum_{u \in L} O(1) + \sum_{v \in S} O\left(1 + \deg(v) + \sum_{x \in N(v)} \deg(x)\right).$$

Notice that during the traversal of $N_2[v]$, each node $w \in N_2[v]$ is marked as covered. To prevent redundant inner explorations, once a node w is marked, subsequent outer iterations skip w. Furthermore, in any graph:

$$\sum_{v \in V} \deg(v) = 2(n - 1), \quad \sum_{v \in V} \sum_{x \in N(v)} \deg(x) = \sum_{x \in V} \deg(x)^2.$$

By marking nodes in $N_2[v]$ and flagging visited vertices, no edge is traversed more than a constant number of times across the entire algorithm execution. Therefore:

$$\sum_{v \in S} O\left(1 + \deg(v) + \sum_{x \in N(v)} \deg(x)\right) = O(n).$$

Hence, $T_{\text{greedy}}(n) = O(n)$, yielding a total running time of:

$$T(n) = O(n) + O(n) + O(n) = O(n).$$

Space Complexity Analysis
The space footprint consists of:

- Adjacency list representation of T: $O(n)$ space.
- Arrays for *depth*, *parent*, and *covered* flags: $3 \times O(n) = O(n)$ space.
- Array L for depth-sorted order: $O(n)$ space.
- Output set S: $O(n)$ space.

Thus, the total space complexity is strictly bounded by $O(n)$.

5 Proof of Correctness

Theorem 1. *Algorithm 1 produces a Minimum Distance-2 Dominating Set S for any rooted tree $T = (V, E)$.*

Proof. Let $S_{\text{alg}} = \{v_1, v_2, \ldots, v_k\}$ be the sequence of vertices selected by Algorithm 1 in order of insertion. Let O be an optimal distance-2 dominating set of minimal cardinality $|O|$. We prove by induction that for every $i \in \{0, 1, \ldots, k\}$, there exists an optimal distance-2 dominating set O_i such that:

$$\{v_1, v_2, \ldots, v_i\} \subseteq O_i \quad \text{and} \quad |O_i| = |O|.$$

Base Case ($i = 0$): Set $O_0 = O$. The empty set $\emptyset \subseteq O_0$, and $|O_0| = |O|$. The hypothesis holds.

Inductive Step: Assume there exists an optimal set O_{i-1} containing $\{v_1, \ldots, v_{i-1}\}$. Consider iteration i, where node $u \in L$ is the deepest uncovered vertex under the partial set $\{v_1, \ldots, v_{i-1}\}$.

According to Algorithm 1, $v_i = \text{ancestor}_2(u)$ (the grandparent $g(u)$, or parent/root if depth < 2). Since O_{i-1} is a valid dominating set, it must contain at least one vertex $w \in O_{i-1}$ such that $\text{dist}_T(u, w) \leq 2$.

We consider two cases:

- **Case 1: $w = v_i$.** Then $v_i \in O_{i-1}$. We set $O_i = O_{i-1}$, satisfying $\{v_1, \ldots, v_i\} \subseteq O_i$ with $|O_i| = |O|$.
- **Case 2: $w \neq v_i$.** Because u is the deepest uncovered node in L, all descendants of u are already covered prior to step i. Therefore, w cannot be a proper descendant of u. The candidate set for w is restricted to $N_2[u] \setminus \text{descendants}(u)$.

  By construction, v_i is the highest possible ancestor of u within distance 2. Any vertex $y \in V$ that is covered by w and was not already covered by $\{v_1, \ldots, v_{i-1}\}$ must lie at depth $\leq \mathit{depth}(u)$. Since v_i is at a higher or equal level in the tree than w, the set of newly covered vertices satisfies:

  $$N_2[w] \setminus \bigcup_{j=1}^{i-1} N_2[v_j] \subseteq N_2[v_i] \setminus \bigcup_{j=1}^{i-1} N_2[v_j].$$

  We define a new candidate set $O_i = (O_{i-1} \setminus \{w\}) \cup \{v_i\}$. Because $N_2[v_i]$ covers every previously uncovered node that $N_2[w]$ could cover, O_i remains a valid distance-2 dominating set for T. Furthermore, $|O_i| = |O_{i-1}| - 1 + 1 = |O_{i-1}| = |O|$.

By structural induction up to $i = k$, there exists an optimal set O_k containing S_{alg}. Since S_{alg} covers all vertices in V, no proper subset of S_{alg} can be valid, implying $|S_{\text{alg}}| \leq |O_k| = |O|$. Since $|O|$ is optimal, $|S_{\text{alg}}| = |O|$. Thus S_{alg} is optimal. □

6 Python Implementation Code

GitHub Repository Link: `https://github.com/SubhojitMaji/DAA-Project-Report`

Below is the clean source code implementation of the algorithm.

```python
from collections import deque
import sys

def min_distance_2_dominating_set(n, edges, root=0):
    if n == 0:
        return 0, []
    if n == 1:
        return 1, [0]

    # Build adjacency list
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)

    # Compute depth and parent array using BFS
    parent = [-1] * n
    depth = [0] * n
    visited = [False] * n
    order = []

    queue = deque([root])
    visited[root] = True

    while queue:
        curr = queue.popleft()
        order.append(curr)
        for nbr in adj[curr]:
            if not visited[nbr]:
                visited[nbr] = True
                parent[nbr] = curr
                depth[nbr] = depth[curr] + 1
                queue.append(nbr)

    # Reverse BFS order gives nodes sorted by depth descending
    order.reverse()

    covered = [False] * n
    S = set()

    # Function to mark nodes within distance 2 as covered
    def mark_covered_dist_2(start_node):
        covered[start_node] = True
        # Distance 1
        for d1 in adj[start_node]:
            covered[d1] = True
            # Distance 2
            for d2 in adj[d1]:
                covered[d2] = True

    # Greedy choice processing
    for u in order:
        if not covered[u]:
            # Select highest available ancestor up to distance 2 (
```

```
                grandparent)
            chosen = u
            if parent[chosen] != -1:
                chosen = parent[chosen]
                if parent[chosen] != -1:
                    chosen = parent[chosen]

            S.add(chosen)
            mark_covered_dist_2(chosen)

    return len(S), sorted(list(S))

# Example usage
if __name__ == "__main__":
    # Sample Tree Input: 7 nodes
    # Edges: (0,1), (0,2), (1,3), (1,4), (2,5), (2,6)
    n_nodes = 7
    tree_edges = [(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)]
    size, team = min_distance_2_dominating_set(n_nodes, tree_edges)
    print(f"Minimum Team Size: {size}")
    print(f"Selected Team Members: {team}")
```

7 Conclusion

We have presented an optimal linear-time algorithm to solve the problem. By proving that the grandparent greedy choice dominates all local alternatives for uncovered deep nodes, we established both linear runtime $O(|V|)$ and strict global optimality. The implementation and tests confirm the theoretical bounds.